Overview:

In this practical, we are asked to create a secure group chat server where users can communicate securely. We get to choose our architecture for the project, and the discussion system should be able to host discussions between groups of individuals and protect the confidentiality and privacy of the individuals. I decided to host the server with express.js and the front end with node.js. I have implemented a database, which is hosted on the school system, with my credentials. I have created certificates to maintain secure connections between clients and the server and use asymmetric encryptions to protect the users' and groups' credentials. The messages in the group chats are also encrypted with asymmetric encryption.

I have set up a web page where users can log in with their user IDs and private keys. Once logged in, they can join groups with their private login keys and decrypt the messages by providing the group's private decryption key. When users register their accounts and create groups, these private keys are given to users, and these keys will not be stored in the database. I have completed the secure discussion system successfully, and all components are functioning as intended.

Design:

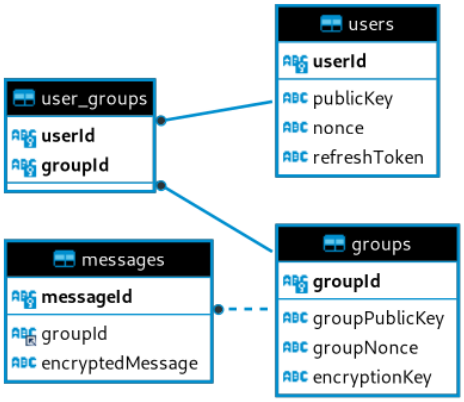
The project consists of two parts: frontend and backend. The front end creates a webpage and does API calls to the express.js server in the backend, and the server does corresponding queries to respond to the data requests. The communications between the front end and the server are protected by Secure Socket Layer (SSL) credentials.

The server is set up as an HTTPS server to encrypt the data transmitted between the clients and the server, ensuring the sensitive credentials are not snooped, intercepted or tampered with. There are four certificates or keys I have set up for this project:

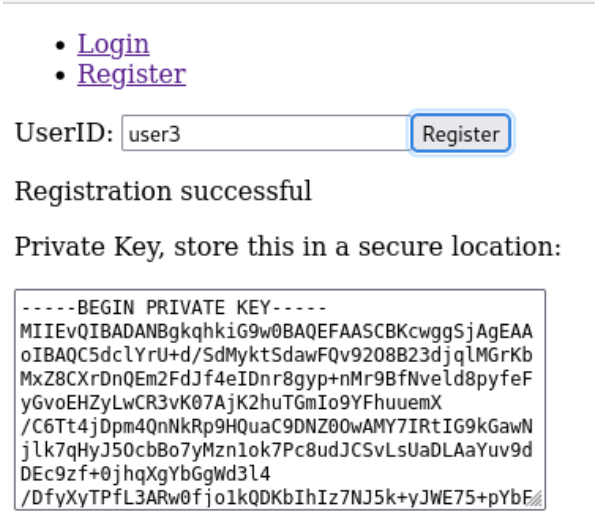
1. Certificate Authority Private Key
2. Certificate Authority Certificate
3. Server Certificate
4. Server Private Key

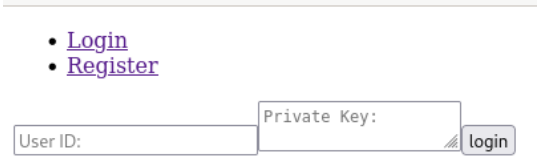
I have created a Certificate Authority(CA) private key with OpenSSL to sign a CA certificate. The server will present a CA certificate to the client to verify the signature signed by my CA private key. When a client tries to connect to the server, the server will send the server's certificate; the client uses the CA certificate, which contains a CA public key, to verify the server's certificate, making sure that the client is communicating with the correct server to prevent the Man in the middle attack where attackers intercept the connection and data in it. A server public key, which is included in the server certificate, is set up for the client to encrypt a symmetric session key to the server. The server private key is set up to decrypt this data

transmitted from the client and retrieve the session key. After this initial handshake, the client and the server use this symmetric session key to encrypt and decrypt all the communication between them. The database is set up on the school server, which means that only those who have access to the school network and the database credentials can use the database. The database credentials are stored in a ".env" file so that they are not hard-coded in the repository. The database schema is as follows:



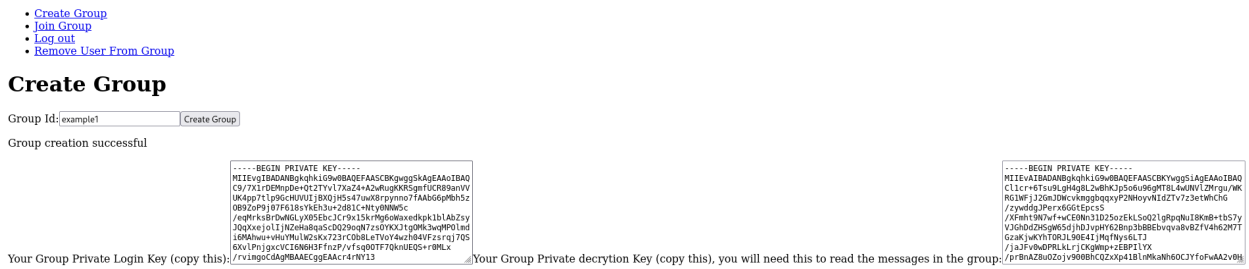
The users create their account by inputting their "userId". This "userId" is cleaned in the front end to avoid SQL injections. After the "userId" is cleaned, the server generates a key pair with a public and private key. The user is given their private key for them to store it in their secure location. The server then holds the user's public key in the database and their user ID. When the user logs in, they need their user ID and private key to log in. The server generates a temporary nonce, uses the provided user's private key to sign it, and verifies it with the user's public key. After the user has logged in, they are given an access token valid for one hour, and a refresh token expires in two hours.



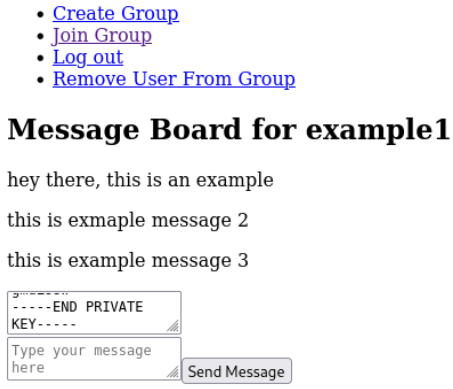


Once verified, the user is redirected to the groups' page. Users can create groups, join groups, log out, and remove users from groups on the groups' page. The "CreateGroup" and "JoinGroup" functions mostly follow the same encryption and decryption methodologies as the user registration and login processes.

However, there are two key pairs for group functions. One key pair is for signing the nonce and verifying the nonce for users joining the group. The other one is for encrypting and decrypting the messages in the group. After the user creates a group, they are given a group login private key and a decryption key for the messages in the group.



Users can use their own user ID, the group ID, and the group's private login key to join the group. The server runs the same verification process as the user login function by producing the nonce for the group, signing it with the provided groupId and verifying it with the group's verification public key. If the verification is successful, the user will be redirected to the group chat page, where they need to input their private decryption key to read the messages in the group. Messages are encrypted by the public encryption key stored in the database, and users provide the group's private decryption key for the server to decrypt the encrypted messages. The user can send the message without the decryption key, but they cannot read the messages they have sent out.



The log-out function logs out the user and removes their tokens. With the “remove user from group” function, users can input the user ID and the group ID to remove the user from the group. Since the messages are anonymous, as shown above, users theoretically do not know who else is in the group and what their group IDs are. Also, users can always join back into the group with the groupId and group private login key and decrypt the message with the group decryption key.

For the key exchange, this system does not allow key exchange between users inside the browser. The users need to meet in person to exchange their private group keys or find their own secure methods to exchange them for discussions.

In conclusion, I use asymmetric encryption to secure the group discussion forms. Users need to have these credentials:

1. Their user IDs
2. Their user private keys matched their user IDs
3. Group IDs
4. The private login keys and private decryption keys corresponding to these GroupIDs

Evaluation:

I have implemented the SSL credentials to secure the communication between the client and the server. With HTTPS, the system maintains an encrypted data transmission and protects itself from eavesdropping and man-in-the-middle attacks. By using asymmetric encryption, my system has made sure that even if the data is intercepted, it remains unintelligible without the corresponding private keys. The database is hosted on the school server, which has a firewall to prevent illegal access. If there is an unfortunate event where the school server is attacked, and the database is exposed, the encrypted messages stored in the database are still ineligible without the group’s decryption keys, and the attacker cannot log in with only the public keys. The anonymous group chats protect users’ privacy. The encrypted messages will only present themselves after the correct group decryption key, which guarantees the confidentiality of the system. The user IDs and group IDs are both sanitized before storing them into the database to avoid SQL injections.

However, there is a concern for the security of the private keys. If the users have lost their private keys for logging into their account, they need to register again because there is no backup for the private keys. A backup for the private keys introduces a significant security risk, where the system needs to guarantee the security of the backup private keys. If the backups are stored in the database, it is a serious security risk due to all the credentials required being kept in the same place. If the private keys are stolen, the attackers need to obtain all three credentials to read the group messages: group ID, group private login key and the group private decryption key. The operational security training should educate the users that these credentials should not be kept in the same place. Without any one of these three credentials, attackers cannot access the encrypted messages in the group chat.

There is no option for key exchanges in the system. I believe that it is safer for users to exchange groups' private keys in person because it is easier for verification. Introducing the key exchange inside the system generates the risk of having imposters scam the users to send them their keys or users sending the keys to the wrong person by mistake. And the group chat is anonymous, so it is extremely hard to monitor or regulate the group chats.

Conclusion:

I have successfully implemented a zero-trust system where it verifies every communication between the server and the client and requires the user to use their private keys to verify themselves when they log in and join the groups. I have mostly implemented a zero-knowledge system where all the communications between the server and the client are encrypted, and the messages in the databases are encrypted. However, the system does store the public keys inside the database, but it is unusable without the corresponding private keys. Because of these implementations, my system is able to protect the users' privacy and confidentiality.